Firmware Architecture Baseline

Hydroponics Monitoring System – ESP32-S3-WROOM-1-N16R8

Revision 3 – 22 October 2026 – project baseline. Supersedes Revision 2 (4 October 2026).

Item	Value
Document role	Architecture baseline for the whole firmware project. Every implementation, test and design change refers to the section, requirement (R), decision (D) and evidence (L/E) identifiers used here.
Target hardware	ESP32-S3-WROOM-1-N16R8 (dual-core Xtensa LX7, 16 MB flash, 8 MB octal PSRAM) as drawn in the Sensor ↔ ESP32-S3 Signal Pinout of 22 October 2026, the Power Circuit Diagram and Redesign Specification Rev 5.
Software baseline	PlatformIO espressif32@6.12.0, Arduino-ESP32 2.0.17 (board esp32-s3-devkitc-1, QIO flash + OPI PSRAM). Project code in C (gnu11) on the framework's C HAL and FreeRTOS kernel. NimBLE-Arduino 2.5.1, ArduinoJson 6.21.5.
Status	Implementation has started (see firmware/README.md). Nothing here is hardware-validated unless stated; timing numbers are design budgets or acceptance targets unless identified as datasheet limits.
Preserved	The inherited application, BLE demo and handover files are not edited by this document.

Change log

Revision	Changes
Rev 1	Architecture decision baseline: requirements R1–R10, decisions D01–D13, evidence L1–L13 and E1–E12.
Rev 2	Same technical content, nothing removed. Restructured for readability: “Architecture at a glance” page, glossary, an “In short” summary per section, nine diagrams, worked examples (50 °C alarm, USB unplugged during a pH read, voltage-divider recovery, example safety rule). Added: D14 (Safety on core 0 must be verified under radio load, with a defined fallback), D15 (single acquisition worker until stage 6), a new acceptance test for Safety latency under radio load, an open-items table, and a framework-version note on the references. Additions are marked with a purple **NEW IN REV 2** box or a purple table row.
Rev 3	Hardware update; structure unchanged. Target is now the ESP32-S3-WROOM-1-N16R8 (D16 supersedes D02); project code is C (D17); the MQ-7 is replaced by an SGP30 on the retained 3.3 V supply (D18). Signal map from the 22 October pin drawing, including the power sense (GPIO 8) and relay drive (GPIO 15); rail names from Redesign Specification Rev 5. New boxes and rows are green (**NEW IN REV 3**); S3, SGP30 and rail-name wording changes run throughout and are not individually marked.

Contents

1. Architecture at a glance . . . 3
2. How to read this document . . . 4
3. Requirements and interpretation . . . 7
4. Inherited code: keep, adapt or replace . . . 8
5. Hardware boundary and pin contract . . . 9
6. Interrupts and realistic alarm latency . . . 10
7. Layered code instead of a large main . . . 12
8. Task and core allocation . . . 13
9. One-second schedule and timing budget . . . 15
10. Data model, ownership and freshness . . . 16
11. Synchronization and queue contracts . . . 18
12. Safety policy and control arbitration . . . 19
13. Power-mode state machine . . . 21
14. Rail races and safe ownership handover . . . 22
15. Low-power scope and framework choice . . . 24
16. Sensor contracts and bring-up decisions . . . 25
17. BLE, MQTT and configuration boundary . . . 27
18. Fault tolerance, deadlines and recovery . . . 28
19. Implementation stages and acceptance . . . 29
20. Decision log and remaining gates . . . 30
21. Local evidence register . . . 32
22. Manufacturer and software references . . . 33

1. Architecture at a glance

IN SHORT

Each sensor has its own driver file. One acquisition task on core 1 reads all sensors in turn every second (the BME280 every 250 ms), checks every value the moment it is read, and publishes a copied “latest snapshot”. Safety, power management, Wi-Fi/MQTT, BLE and configuration run on core 0 and never wait for a sensor. When external power disappears, a second acquisition task on core 0 takes over the four 3.3 V sensors and core 1 goes idle.

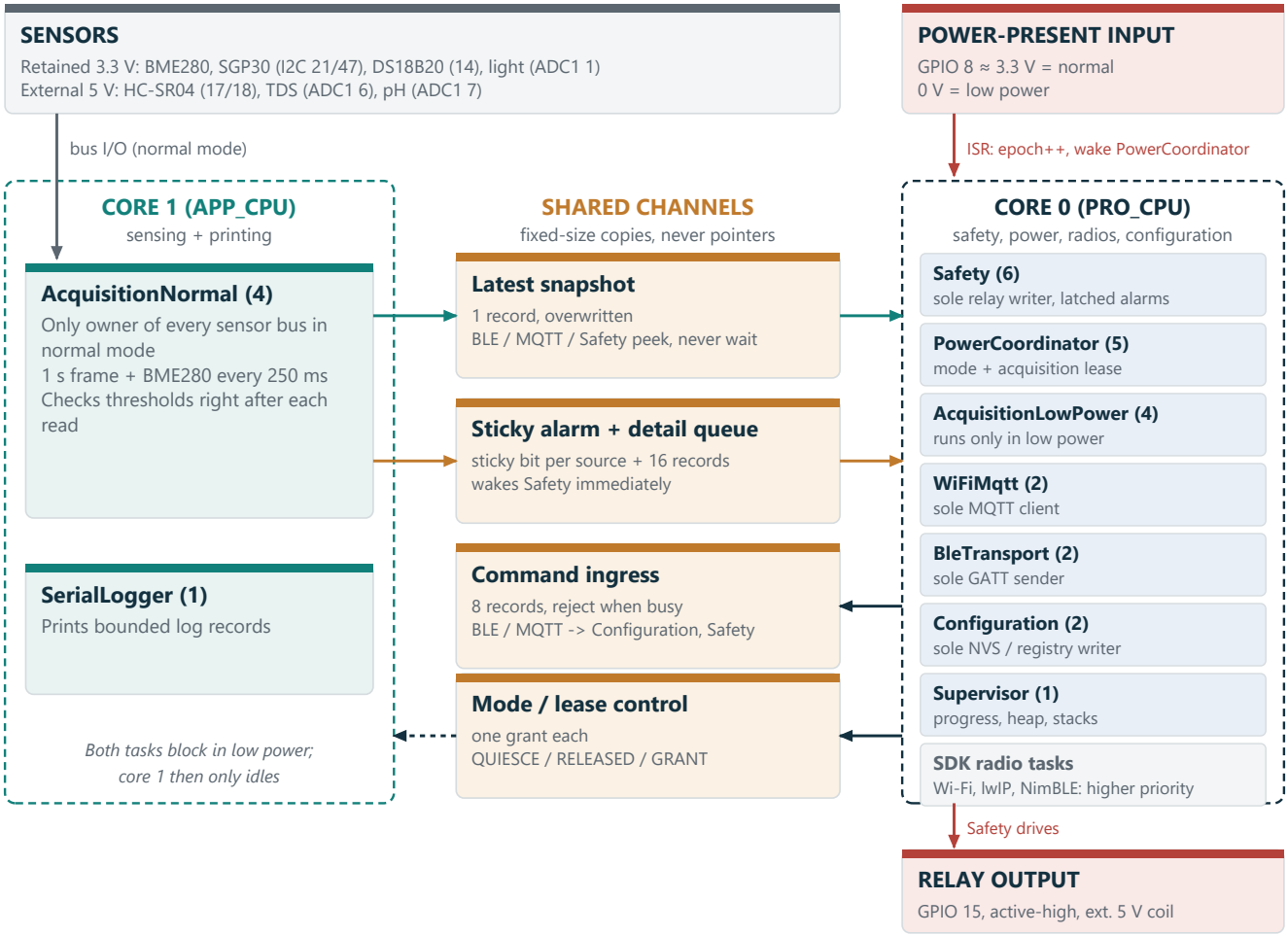


Figure 1 – Tasks, cores and the only channels through which they share data. Numbers in brackets are FreeRTOS priorities (higher = more urgent).

The six rules that hold the design together

1. **One owner per resource.** Exactly one task touches each sensor bus, the relay output, the MQTT client, the GATT server and NVS. Nobody else calls them; they send a request instead.
2. **Copy, never share.** Tasks exchange fixed-size copied records through FreeRTOS queues. No pointers, Arduino `String` buffers or JSON views cross a task boundary.
3. **Readers never wait for sensors.** BLE and MQTT read the latest snapshot immediately and report its age; a refresh request is asynchronous.
4. **Check immediately, act locally.** Each valid sample is compared against its thresholds right after it is read; a critical value wakes Safety at once. Network reporting is secondary.
5. **Power is a versioned fact.** A rail epoch counter (not only a boolean) tags every 5 V measurement, so a reading taken across a power cut is discarded instead of becoming a false alarm.

6 **Everything is bounded.** Every wait has a timeout, every queue has a size and an overflow policy, every task is supervised by a watchdog or progress counter.

Four decisions to understand first

- Use the ESP32's supported FreeRTOS kernel explicitly. Creating our own tasks and schedules does not mean installing a second RTOS. Replacing the Espressif kernel would be a different project and conflicts with the selected Arduino/Wi-Fi/BLE stack.
- A BME280 has no threshold interrupt pin. Software can prioritize a critical value only after a measurement reveals it. Faster sampling reduces that blind interval; an independent threshold sensor is needed to eliminate dependence on polling.
- A one-second acquisition schedule is not a promise that every physical sensor produces a new, accurate value every second. SGP30 warm-up and baseline learning, DS18B20 conversion time and probe settling remain physical constraints.
- Low power follows the circuit's rails: BME280, SGP30, DS18B20 and DFR0026 stay on the retained 3.3 V supply; HC-SR04, pH and TDS lose their external 5 V supply and become unavailable. BLE remains available. The relay coil is fed from external 5 V, so the relay cannot be energized on battery.

The proposed normal-mode allocation is acquisition and serial logging on core 1, with Wi-Fi, MQTT, BLE, power coordination and short safety actions on core 0. In low power, only the core-0 acquisition worker is active; the normal worker and logger remain blocked. This is single-core *application* execution, not runtime power removal from one CPU.

2. How to read this document

IN SHORT

Sections 3–5 set the requirements, what we keep from the old code, and the hardware limits. Sections 6–14 define how the firmware works. Sections 15–19 cover low power, drivers, communications, faults and the delivery plan. Sections 20–22 hold the decision log and the sources behind every claim.

Sections	Topic	Read it when you…
3–5	Requirements, inherited code, hardware/pin contract	need to know why something is required or which pin/rail a sensor uses
6–8	Interrupt limits and alarm latency, code layers, tasks and cores	write a driver, create a task, or wonder where code belongs
9–12	1 s schedule, data model, synchronization, safety policy	touch the acquisition loop, a queue, or an alarm rule
13–15	Power state machine, rail races and lease handover, low-power scope	work on low-power mode or anything on the external 5 V branches
16–18	Sensor contracts, BLE/MQTT/configuration, fault tolerance	bring up a sensor, change the protocol, or handle an error
19–22	Stages and acceptance tests, decision log, evidence and references	plan work, review a change, or check a source

Conventions

- **R1–R10** requirements (section 3), **D01–D18** decisions (section 20), **L1–L14** local evidence files (section 21), **E1–E13** manufacturer/software references (section 22). Brackets such as [L8, E2] point to the evidence behind a statement.
- "Budget" or "target" = a design number to be verified by measurement. "Datasheet limit" = a manufacturer figure.
- Priorities are FreeRTOS priorities: a higher number is more urgent. Stack sizes are in bytes as used by the ESP-IDF task API (1 KiB = 1024 bytes).
- GPIO numbers are ESP32-S3 GPIOs, not module pad numbers.
- Boxes: **In short** = plain summary; **Note** = caution; **Worked example** = step-by-step scenario; **New in Rev 2** and **New in Rev 3** = added content.

Glossary

Term	Meaning in this project
ADC1 / ADC2	The ESP32-S3's two analog-to-digital converters. ADC2 is shared with Wi-Fi, so every analog input is on ADC1 (GPIO 1–10). This board uses GPIO 1, 6, 7 and 8.
Attenuation	ADC input scaling. At the highest setting (12 dB, called 11 dB in older APIs) the ESP32-S3 measures up to about 3.1 V; signals above that saturate [E4].
APP_CPU / PRO_CPU	ESP32-S3 core 1 / core 0. Espressif's radio stacks normally run on core 0 (PRO_CPU).
Task	An independent FreeRTOS thread with its own stack, priority and (here) fixed core.
Task notification	A lightweight FreeRTOS signal that wakes one specific task. It carries no measurement and cannot discover a sensor value.
Queue	A FreeRTOS FIFO that copies fixed-size records between tasks. A length-one queue written with xQueueOverwrite() always holds only the newest record.
Snapshot	The latest complete copy of all readings with their quality flags, ages and sequence numbers.
Sticky alarm	An alarm bit that stays set until Safety handles it. A later normal reading cannot erase it.
Lease / generation	Permission for exactly one acquisition worker to use the sensor buses. Each grant has a generation number so stale acknowledgements are recognized and ignored.
Quiesce	Ask a task to stop starting new work, finish or cancel what is in progress, and acknowledge.
Rail epoch	A counter incremented by the power ISR on every power-sense edge. A measurement is valid only if the epoch is unchanged from start to end.
ISR	Interrupt service routine: minimal code that runs on a hardware event. No printing, I2C, allocation, JSON, Wi-Fi or delay calls.
Forced mode	BME280 mode in which the host triggers one measurement, the sensor converts, then returns to sleep.
Oversampling ×1	The BME280 takes one sample per measurement for temperature, pressure and humidity (written "x1/x1/x1" in Bosch tables).
IIR filter	BME280 internal smoothing filter; disabled initially so a step change is not delayed.
WCET	Worst-case execution time: the longest a piece of code can take.
Jitter	Variation of the actual start time of a periodic job around its ideal time.
Monotonic clock	A time counter that only increases (e.g. microseconds since boot). Used for ages and deadlines, unlike NTP wall-clock time.
Task watchdog (TWDT)	ESP-IDF watchdog that reports or resets if a subscribed task stops feeding it.

Term	Meaning in this project
WAITI / idle	When a core has no runnable task, its idle task executes the WAITI instruction and the core waits clock-gated. It is not powered off.
DFS / light sleep	Dynamic frequency scaling and automatic light sleep; require an SDK built with power management (CONFIG_PM_ENABLE).
Unicore	CONFIG_FREERTOS_UNICORE: a build option that runs FreeRTOS on one core only. Cannot be switched at runtime.
NVS	Non-volatile storage in flash (used by Preferences) for configuration.
GATT	The BLE attribute protocol: services and characteristics (State, Telemetry, Command, Result, Auth).
RMT	ESP32-S3 remote-control peripheral, usable to time the HC-SR04 echo pulse in hardware.
CRC	Checksum. The DS18B20 scratchpad carries one, and the SGP30 adds a CRC-8 to every 16-bit word, to detect corrupted transfers.
TVOC / CO2eq	SGP30 outputs: total volatile organic compounds in ppb and an *estimated* CO2-equivalent in ppm derived from the same signal. CO2eq is not a CO2 measurement [E13].
SGP30 baseline	The sensor's internal compensation state. It is read back periodically, stored, and restored after a restart so readings do not have to re-learn it [E13].
Hysteresis / debounce	Different trip and clear thresholds / a required persistence time, both to prevent chatter.
Backoff	Increasing wait between retries (here 1, 2, 4, 8 … up to 60 s with jitter).
High-water mark	The minimum free stack a task has ever had; used to size stacks from measurement.
PSRAM	External RAM. The N16R8 module embeds 8 MB octal PSRAM, which occupies GPIO 35–37 [L14]. Task stacks, queues and ISR data stay in the 512 KB internal SRAM.
Strapping pin	A pin sampled at reset to select boot mode or flash voltage. On the ESP32-S3: GPIO 0, 3, 45 and 46 [L14].

3. Requirements and interpretation

IN SHORT

Ten requirements come from the project request. Each has an architecture response and a way to accept it. The two most often misread are “sequential” (one task talks to the sensors at a time, but conversions may overlap) and “40 or 50 degrees” (examples, not commissioned thresholds).

ID	Requirement	Architecture response / acceptance
R1	Explicit RTOS synchronization	FreeRTOS tasks, bounded queues, notifications and one owner per peripheral. No application work in Arduino loop().
R2	Sequential polling every second	Absolute one-second frame; one acquisition worker serializes driver operations. Conversion waits yield the CPU.
R3	Critical readings receive priority	Check each completed valid sample immediately; latch critical events and wake Safety. BME280 also runs every 250 ms.
R4	Sensing and networking on different cores	Normal acquisition on APP_CPU/core 1; network application tasks on PRO_CPU/core 0. Configure and inspect stack affinity.
R5	BLE must not wait for sensors	Read a copied latest snapshot; return its age, quality and availability immediately. A refresh request is asynchronous.
R6	Approximately 3 V means normal; 0 V means low power	Dedicated external-power-present input on GPIO 8 (220 Ω over 2 × 220 Ω from external 5 V, ≈ 3.33 V nominal), with ISR invalidation and debounce. It is not a battery-voltage measurement.
R7	Low power retains 3.3 V sensors and BLE	Disable external-5 V measurements and Wi-Fi/MQTT; hand acquisition of BME280, SGP30, DS18B20 and DFR0026 to the core-0 worker; keep core-1 application tasks blocked.
R8	Fault tolerance	Timeouts, independent safety state, stale-data detection, bounded storage, progress monitoring and explicit recovery.
R9	Driver-by-driver development	Dedicated PlatformIO bring-up environments use the same production drivers and a small test main.
R10	Detailed plan before sensor implementation	This PDF and editable source define interfaces, decisions and tests. Driver coding is the next stage.

What “sequential” means

Only one worker issues sensor transactions at a time. Start a conversion, return control, service another sensor and later collect the result. Sensors may convert internally at the same time; their bus transactions remain serialized. Waiting for every conversion to finish before touching another sensor would waste the one-second budget (the DS18B20 alone needs up to 750 ms).

Planning assumptions

Retain PlatformIO and the Arduino-ESP32 framework from the submitted draft; project code is written in C (D17). Keep the baseline one-second schedule in both modes for enabled sensors. The BME280 fast interval also remains 250 ms in both modes; relaxing it for power savings requires a changed detection-latency requirement. BLE state/telemetry transport intervals are separate from sensing intervals.

THRESHOLDS ARE NOT YET DEFINED

Temperature values such as 40 or 50 degrees in the request are examples, not commissioned safety thresholds. Define real limits, hysteresis, actuator actions and reset rules before enabling automatic control. [L1–L4]

4. Inherited code: keep, adapt or replace

IN SHORT

The old firmware is one Arduino loop in which slow network retries and blocking sensor reads delay each other, and drivers publish straight to MQTT. We start a new firmware/ directory, keep the old code untouched for reference, and port only algorithms whose units, wiring, timing and calibration have been checked.

Evidence in the current files	Decision and reason
Handover src/main.cpp:39, 72–117	Replace the superloop/reconnect orchestration. HTTP GET and retry loops execute on the same path as sensing; MQTT retries delay and eventually restart the device.
src/ultraSonicSensor.cpp:25	Replace default-timeout polling with capture + a sensor-specific deadline. pulseIn() is called without an explicit timeout (the 1 s default applies).
src/waterTemp.cpp:11–15	Retain the DS18B20 concept, replace its scheduling. It waits 100 seconds between updates and uses requestTemperatures() without asynchronous configuration.
src/sensor.cpp:15–32	Remove MQTT publishing from drivers. Drivers return observations; transport services serialize and send them.
src/tdsSensor.cpp:36–65	Rework sampling and compensation. One sample is inserted every 8 seconds, publication is every 5 seconds, and the initial buffer contains zeros. Comments disagree with executable intervals.
src/phSensor.cpp:16–17	Reuse only as historical reference. Raw ADC scaling and hard-coded coefficients must become calibrated conversion stages, including the actual divider.
deviceManager.cpp / deviceFactory.cpp	Keep stable device identities and supported-type concepts. Replace unrestricted runtime allocation and arbitrary pin changes with a bounded registry and validated board profiles.
ESP-Demo/src/Main.cpp:202–213	Reuse the intent of setLowPower(): request Wi-Fi shutdown/restart and publish mode. Move actual changes into the coordinated services.
ESP-Demo/Main.cpp:451, 524, 536	Keep callback-to-task command dispatch, but use a bounded pool or fixed records with explicit ownership, overflow results and session identity.
FirmwareContract.md	Retain UUIDs, framing and authenticated commissioning concepts. Revise physical health, power policy and capabilities before connecting real sensors.

Recommended migration strategy

Create a separate `firmware/` directory when implementation starts. Do not delete the old team code. Port algorithms only when their units, electrical assumptions, timing and calibration have been verified. Retain compatibility adapters for useful backend topic names and BLE operations instead of keeping transport code inside sensors.

The inherited climate implementation uses a DHT-family library; it is not a BME280 driver. The demo has no real sensor or relay drivers and reports a fixed simulated battery percentage. Neither is a production implementation of this plan. [L2–L4]

5. Hardware boundary and pin contract

IN SHORT

Four sensors (BME280, SGP30, DS18B20, DFR0026) run from the retained 3.3 V supply and stay alive on battery. The HC-SR04 and the relay coil use the external 5 V supply, and pH and TDS a filtered branch of it; all three lose power when external power is removed. All analog inputs use ADC1 so they keep working with Wi-Fi on. The power-sense input (GPIO 8) and the relay drive (GPIO 15) are now on the drawing; their features stay disabled until the hardware behavior has been measured.

The Power Circuit Diagram is the topology authority. The Sensor ↔ ESP32-S3 Signal Pinout (Pin Layout, 22 October 2026) defines the signal map, and Redesign Specification Rev 5 §3 restates it. These are design inputs, not proof that the physical assembly matches. [L5–L7]

Device / function	Supply branch	GPIO / interface	Firmware consequence
BME280	Retained 3.3 V	SDA 21, SCL 47; I2C 0x76	Forced measurement; verify chip ID. Shares the bus with the SGP30; 4.7 kΩ pull-ups to 3.3 V.
SGP30 module	Retained 3.3 V	SDA 21, SCL 47; I2C 0x58	Measurement every 1 s, CRC on every word, WARMING for the first 15 s. The module must accept 3.3 V supply and logic.
DS18B20	Retained 3.3 V	DQ 14; 4.7 kΩ pull-up	Externally powered 1-Wire; asynchronous conversion and CRC.
DFR0026 light	Retained 3.3 V	1 / ADC1_CH0; 10 kΩ series, 1 µF	Raw/mV first; do not invent a lux conversion. The RC filter ($\tau \approx 10$ ms) limits how fast the input can follow.
SEN0244 TDS	Filtered external 5 V	6 / ADC1_CH5; 100 nF	0–2.3 V nominal module output; water-temperature dependency.
pH-4502C	Filtered external 5 V	7 / ADC1_CH6; 10 kΩ / 15 kΩ, 1 µF	Divider correction before calibration; DO/TO unconnected.
HC-SR04	External 5 V	TRIG 17, ECHO 18	1 kΩ / 2 kΩ ECHO divider; capture timing; no-echo timeout. TRIG is driven at 3.3 V logic.
External-power present	External 5 V (sense)	8 / ADC1_CH7; 220 Ω over 2 × 220 Ω	≈ 3.33 V at 5.00 V input. Digital input with an edge ISR; the ADC can confirm levels during commissioning.
Relay drive	External 5 V coil	15; IRLZ44N low side, 220 Ω gate, 10 kΩ pull-down, 1N5822 flyback	Active-high. The pull-down holds the coil off through reset and boot. One SRD-05VDC relay; no coil supply on battery.
Native USB	USB	D− 19, D+ 20; 22 Ω series	Programming and USB-Serial-JTAG console. Never reassign.
Branch-health feedback	–	Not on the drawing	GPIO 8 senses the external feed, not the health of each 5 V branch.

Electrical limits that affect code

- ADC1 (GPIO 1–10) avoids the ADC2/Wi-Fi conflict. GPIO 1, 6, 7 and 8 are used; GPIO 2, 4, 5, 9 and 10 remain free ADC1 inputs.
- GPIO 35–37 belong to the octal PSRAM on the N16R8 and must not be used. GPIO 19/20 are the native USB pair. The strapping pins are GPIO 0, 3, 45 and 46: the drawing puts the BOOT button on GPIO 0 and leaves 3, 45 and 46 unconnected. Review them before assigning anything new. [L6, L14]

- Recover module voltage as $V_{module} = V_{adc} \times (R_{top} + R_{bottom}) / R_{bottom}$. Nominal factors are 25/15 for pH, 3/2 for the HC-SR04 echo and 660/440 for the power sense. Measure actual ratios.
- At its highest attenuation the ESP32-S3 ADC measures up to about 3.1 V [E4]. A 5 V pH output becomes 3.0 V at the pin: inside the range, but at its top, where non-linearity is largest. The DFR0026 is fed from 3.3 V and can exceed 3.1 V in bright light, which then saturates. Characterize both on the board; "below 3.3 V" is not an accuracy guarantee. [E4, L6]
- The ESP32-S3 input thresholds are $V_{IH} \geq 0.75 \times VDD$ (2.475 V) and $V_{IL} \leq 0.25 \times VDD$ (0.825 V), with V_{IH} at most VDD + 0.3 V [L14]. The power-sense node is ≈ 3.33 V at 5.00 V and ≈ 3.5 V at 5.25 V, close to that ceiling; measure the actual feed voltage.

WORKED EXAMPLE – RECOVERING THE PH MODULE VOLTAGE

The pH divider is 10 kΩ (top) over 15 kΩ (bottom). If the calibrated ADC reads 1.50 V, the module output is 1.50 × (10 + 15) / 15 = 2.50 V. At the module's 5.0 V maximum, the pin sees 5.0 × 15 / 25 = 3.0 V – just inside the S3's ~3.1 V range, so the top of the pH range sits where the ADC is least linear. For the power sense (220 Ω over 440 Ω): 5.00 V at the input gives 5.00 × 440 / 660 = 3.33 V at GPIO 8, well above the 2.475 V logic-high threshold.

NEW IN REV 3 – HARDWARE LABELS RESOLVED

The power diagram and pin drawing now use IRLZ44N for both the source-selection stage and the relay driver, with 1N5822 diodes, and show one SRD-05VDC relay. The earlier "IRLZ440N" label and the IRF4905 of Redesign Specification Rev 4 are superseded. The BLE demo catalog can still describe four relays; the board has one. [L3, L5–L7]

6. Interrupts and realistic alarm latency

IN SHORT

No sensor on this board can raise an interrupt when a value becomes critical, so "priority" is earned by checking every reading immediately and by reading the BME280 four times per second. The worst-case software reaction to a sustained over-temperature is about 277 ms (target 300 ms). Real interrupts are used for power-sense edges and echo timing, not for sensor values.

Observation comes before priority

A task notification is a software scheduling mechanism. It cannot discover a sensor value that has not been measured. The BME280 pin table has no alarm/INT output. The DS18B20 alarm-search command also requires host bus activity after a conversion; it is not an asynchronous interrupt pin. The pH module's DO comparator output is unconnected in the signal drawing, and the SGP30 has no interrupt output. [L6, L8–L9]

Three mechanisms are therefore separate:

1. **Immediate check.** Immediately after each valid reading, evaluate the configured threshold, update a sticky pending-alarm record and notify Safety. Do not wait for the one-second snapshot boundary.
2. **Fast lane.** Poll BME280 every 250 ms using forced mode, oversampling ×1 for temperature, pressure and humidity, and no IIR filter delay initially. Keep ordinary sensors on the one-second schedule.
3. **Independent hardware (optional).** If an independently bounded temperature response is essential, add a suitable temperature switch or sensor with a hardware alert output.

Example latency budget – not a measurement

For a sustained condition observable by the BME280, the proposed software detection/action bound is:

```
250 ms sample interval + 5 ms scheduling target + 12 ms conversion/read budget
+ 10 ms Safety target = 277 ms;  round acceptance target to 300 ms.
```

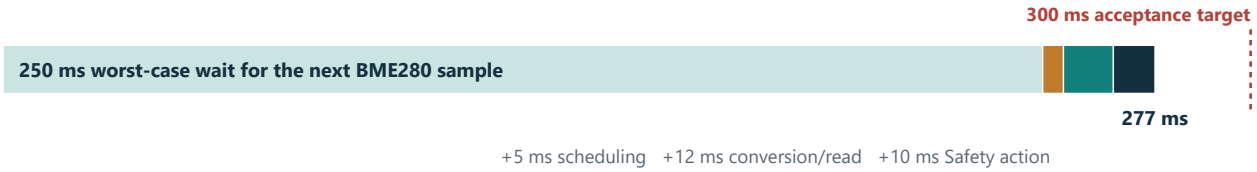


Figure 2 – Worst case: the temperature crosses the limit just after a sample, so it is seen at the next one 250 ms later.

The Bosch maximum conversion formula at oversampling ×1/×1/×1 gives 1.25 + 2.3 + (2.3 + 0.575) + (2.3 + 0.575) = 9.3 ms. The 12 ms slot includes a small bus margin; verify it on the chosen I2C setup. Sensor thermal response, enclosure placement, filtering and actuator mechanics are additional delays. A short excursion entirely between samples may be missed. [L8, section 9.1]

WORKED EXAMPLE – AIR TEMPERATURE RISES TO 50 °C

t = 0 ms – AcquisitionNormal reads the BME280: 20 °C, OK. It continues with DS18B20, HC-SR04 and the analog channels.

t ≈ 1 ms – the air actually crosses the configured limit. Nothing can see it yet: the BME280 has no interrupt pin.

t = 250 ms – the fast-lane deadline interrupts the ordinary order between bounded operations. Forced measurement + read (≤ 12 ms) returns 50 °C.

t ≈ 262 ms – in the same step, the pure threshold function trips: the sticky alarm bit for BME280 temperature is set, a detail record is queued and Safety (core 0, priority 6) is notified.

t ≤ 277 ms – Safety takes the configured local action (≤ 10 ms target). The snapshot is published; BLE and MQTT report the alarm afterwards.

For comparison, the old firmware read climate every 5 s inside a loop that could block for seconds, so the same event could go unseen for 5 s or more.

Genuine interrupt uses

External-power edges on GPIO 8 invalidate measurements immediately and wake PowerCoordinator. HC-SR04 echo edges or RMT capture indicate measurement completion, not a critical-value alarm. Optional comparator wiring needs confirmed voltage levels, polarity, hysteresis and meaningful calibration; the pH module's DO pin is not a calibrated pH alarm.

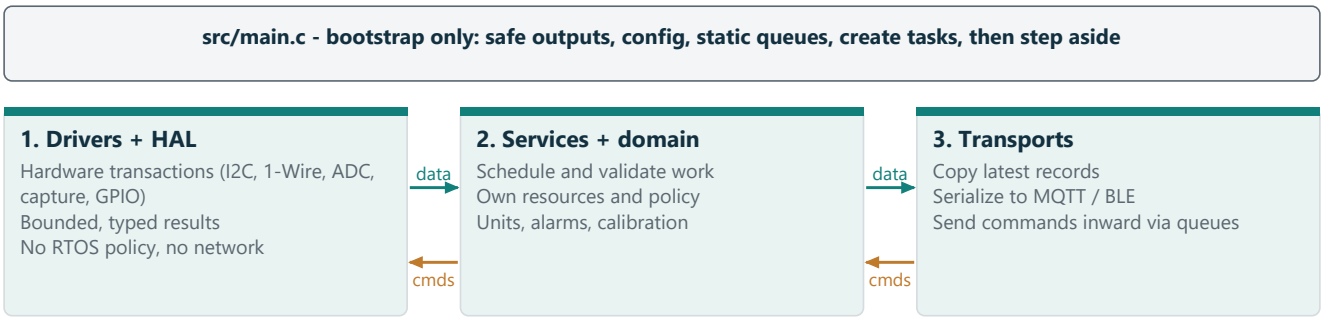
ISRs only capture minimal state and use ISR-safe notification APIs. No printing, I2C, allocation, JSON, Wi-Fi or delay calls. Install the power ISR on core 0 and verify its code/data residency (internal RAM, not PSRAM or flash) and interrupt behavior in the selected SDK. [E2]

7. Layered code instead of a large main

IN SHORT

Keep the habit of one .h/.c pair per sensor – that is the driver layer. What replaces the "big main" is two more layers: services that schedule and decide, and transports that talk to the network. main.c only boots the system. Each sensor also gets its own small test main as a separate PlatformIO environment.

```
firmware/
  platformio.ini            pinned build + test_<sensor> environments
  include/board_profile.h   pins, rails, capabilities; unresolved = disabled
  include/app_config.h      periods, queue sizes, budgets, policy defaults
  src/main.c                bootstrap only
  src/drivers/              bme280, sgp30, ds18b20, light, hcsr04, ph, tds
  src/hal/                  I2C, 1-Wire, ADC calibration, capture, GPIO
  src/domain/               readings, units, alarms, policy, calibration
  src/services/             acquisition, safety, power, configuration
  src/transports/           wifi_mqtt, ble, protocol adapters
  src/diagnostics/          progress counters, reset cause, buffered logs
  test/bringup/             one small entry point per sensor
  test/native/              policy, scheduling, frame and transition tests
```



Sensor data flows right. Commands flow left only through validated queues.

Figure 3 – The three layers below a bootstrap-only main.c.

Rules between layers

- **Drivers** know hardware and report status. They do not publish, select system power mode, own arbitrary RTOS tasks or change a relay as a side effect of reading a sensor. The HAL may internally use bounded SDK waits or capture interrupts; the sensor API does not expose networking or service state.
- **Domain** functions perform unit conversion, validity checks and threshold decisions using explicit inputs (pure functions, testable on a PC under test/native/).
- **Services** schedule work and own resources.
- **Transports** consume copied data and submit commands. No driver includes MQTT/BLE headers or depends on a global client.
- Project code is C (D17): one .h/.c pair per module. A driver is a struct whose first member is the common `SensorDriver`, holding a table of operation pointers, in place of a C++ class. The framework's C++ libraries (NimBLE-Arduino, ArduinoJson) are used only through a thin adapter, or replaced by NimBLE's C host API and cJSON/snprintf. One file pair per sensor is useful; **one task per sensor is unnecessary** for this sequential system.

Thin main responsibilities (in order)

1. Set safe output defaults before network startup.
2. Load and validate board/configuration data.
3. Create static queues and workers.
4. Sample the initial power state.
5. Start Safety and PowerCoordinator; then grant acquisition ownership.
6. Initialize communications without waiting for an access point. Initialization failure enters a reported safe fault state.

Arduino's loop() remains a compatibility stub that blocks indefinitely, with loop-task watchdog subscription disabled/removed explicitly; idle watchdogs remain enabled. No sensing, reconnect or policy work is placed there. Do not remove the framework kernel or blindly delete its loop task. setup() and loop() are written in C and bound to the symbols the Arduino core calls.

8. Task and core allocation

IN SHORT

Nine application tasks: two on core 1 (normal acquisition and the serial logger) and seven on core 0. Safety and power coordination stay on core 0 in both modes so the relay controller never migrates. The low-power acquisition worker already exists on core 0 and simply waits until it is granted the sensors.

These are initial application priorities, not guarantees against higher-priority SDK radio tasks. A higher number is higher priority. Stack reservations are starting budgets in bytes for ESP-IDF task APIs and must be measured with both radios active. [E2]

Task	Core / priority	Trigger and ownership	Initial stack
Safety	0 / 6	Alarm/control notification; 100 ms health check. Sole relay-output owner; short, bounded actions.	3 KiB
PowerCoordinator	0 / 5	Power edges / transition acknowledgements. Owns mode and acquisition lease.	4 KiB
AcquisitionNormal	1 / 4	Absolute deadlines; all sensor buses in normal mode. Blocked in low power.	4 KiB
AcquisitionLowPower	0 / 4	Same engine, 3.3 V devices only. Blocked in normal mode.	4 KiB
WiFiMqtt	0 / 2	Connection events, retries, latest data. Sole Wi-Fi/MQTT application owner.	6 KiB
BleTransport	0 / 2	Bounded RX/TX queues, state and telemetry deadlines. Sole application GATT sender.	6 KiB
Configuration	0 / 2	Validated commands and persistence. Sole registry/NVS writer.	4 KiB
Supervisor	0 / 1	Every 1 s: progress, queue usage, stack/heap and health.	3 KiB
SerialLogger	1 / 1	Bounded log queue; normal mode only. Quiesces in low power.	2 KiB

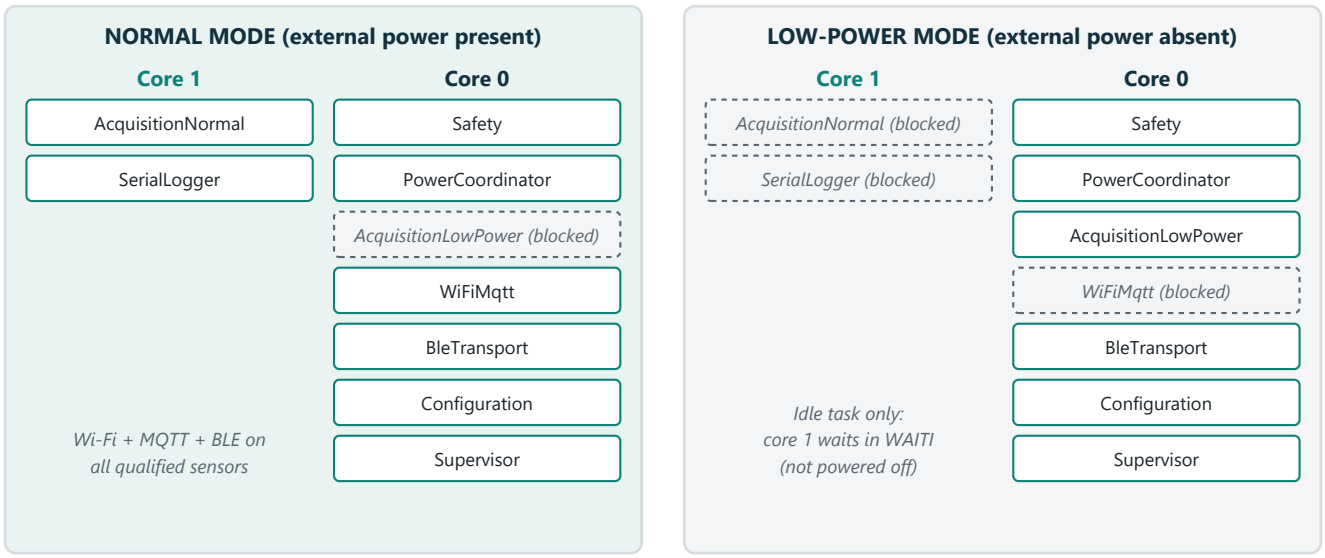


Figure 4 – Which tasks run in each mode. Dashed = created but blocked; it consumes stack, not CPU.

Keeping Safety on core 0 in both modes avoids migrating an actuator controller. Its handler must never wait for network I/O or NVS. Most calculations remain in acquisition on core 1 in normal mode. This is an intentional exception to the simple “all sensing logic on one core” rule.

The two acquisition tasks execute the same engine and share a context only through an exclusive lease. Only the granted worker can access it. The inactive worker blocks indefinitely on a grant queue; it does not poll or touch the drivers. Budget about 36 KiB for these application stacks, plus queues, driver state and substantial SDK/BLE/network memory. Keep stacks, queues and ISR-visible data in internal SRAM. The 8 MB PSRAM suits large buffers such as the telemetry backlog or BLE frame pool, but only after its access latency is measured and nothing in it is touched from an ISR.

Wireless controller and host tasks belong to the SDK/libraries. Confirm their actual affinity; pinning BleTransport does not itself pin the NimBLE host. Configure the host using the selected library's supported build setting and inspect task traces before claiming core 1 has no application work in low power. [E1–E2, E6–E8]

NEW IN REV 2 – D14: SAFETY SHARES CORE 0 WITH THE RADIO STACKS

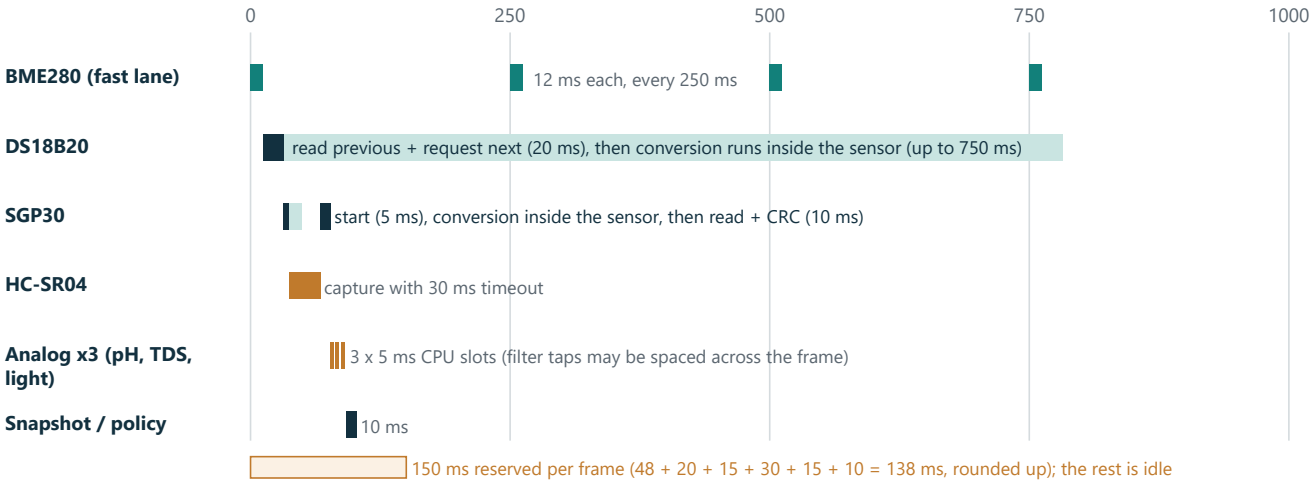
SDK tasks for Wi-Fi, lwIP (TCP/IP) and the NimBLE host also run on core 0, typically at FreeRTOS priorities around 18–23 – well above Safety's 6. A burst of radio work can therefore delay Safety even though it is the most urgent *application* task. The ≤ 10 ms sample-to-Safety-action target (section 19) must be measured **with Wi-Fi and BLE under heavy traffic**, not on an idle radio. Read the actual SDK priorities from a task trace of the resolved build. **Fallback if the test fails:** run Safety on core 1 in normal mode (beside AcquisitionNormal, at a priority above it) and hand it to core 0 only in low power, through the same quiesce/grant protocol as acquisition (section 14).

9. One-second schedule and timing budget

IN SHORT

Every second the active worker runs a frame: collect the DS18B20 result and start the next conversion, then the SGP30, HC-SR04, the analog sensors and bookkeeping, with a BME280 read slotted in every 250 ms. Planned work is about 138 ms per second (150 ms reserved), so the CPU is free most of the time. Slow conversions run inside the sensors while the worker does other things.

The active acquisition worker maintains absolute deadlines using a monotonic clock. It sleeps until the next due event or control notification. vTaskDelayUntil() suits a simple periodic bring-up loop; the integrated multi-rate engine uses a timed notification wait so power changes can wake it immediately. It never runs a one-second delay that hides intervening 250 ms jobs.



Illustrative budget, not a measured trace. Time in milliseconds within one 1-second frame.

Figure 5 – One frame. Dark bars occupy the worker; light bars are conversions running inside the sensor.

Example steady-state sequence

1. At each one-second boundary, read the DS18B20 conversion started in the previous frame and start its next conversion.
2. Service due BME280 conversion/start/read steps.
3. Then the SGP30 measurement (it needs its own steady 1 s cadence), HC-SR04, pH, TDS and light.
4. The BME280 deadline can interrupt this order between bounded operations.
5. A DS18B20 result is absent during the initial conversion; mark it WARMING, never zero.

Work	Proposed budget / period	Meaning
BME280 forced conversion + read	12 ms elapsed, every 250 ms	9.3 ms maximum conversion at oversampling ×1/×1/×1 from the datasheet; time out if not ready.
DS18B20 read, CRC and next request	20 ms host budget / 1 s	Up to 750 ms conversion at 12 bits runs without occupying the worker.
SGP30 measurement + read	15 ms host budget / 1 s	Send the measure command, let the sensor convert while the worker serves other steps, then read 6 bytes and check both CRCs. Confirm the conversion time against the datasheet [E13].
HC-SR04	30 ms capture timeout / 1 s	Use RMT or an audited edge-capture adapter. No echo is TIMEOUT, not zero distance.

Work	Proposed budget / period	Meaning
Three analog channel groups (pH, TDS, light)	5 ms each per frame initially	CPU/ADC budget only; filter window and analog settling are separate.
Publish snapshot / policy / bookkeeping	10 ms per frame	Bounded records; no JSON or serial formatting here.
Normal frame reservation	150 ms per second	48 + 20 + 15 + 30 + 15 + 10 = 138 ms, rounded up (48 = 4 BME280 slots × 12 ms). Not measured WCET or CPU utilization.

Rules inside the frame

- Serial transaction access remains single-owner even while conversions are pending.
- Each service step should finish within 5 ms normally, with documented, bounded timeout exceptions.
- Reserve enough time before the next BME280 deadline; defer an ordinary operation that would cross it.
- Do not retry a failing sensor repeatedly inside the same frame.
- The BME280 and SGP30 share one I2C bus. Their transactions are serialized by the single owner; a BME280 read may run while the SGP30 converts. Check on a logic analyzer that the SGP30 never holds the bus during conversion.
- TDS/pH filter taps may be scheduled as small additional ADC substeps. A five-millisecond CPU budget does not justify compressing a manufacturer's spaced filter window into an instantaneous burst. Start with valid sample counts, record window age, and verify noise/latency before fixing the filter configuration. [L8–L12]

10. Data model, ownership and freshness

IN SHORT

Every value travels with its quality, its age and the power epoch it was measured in. Consumers never guess: a value is OK, STALE, FAULT, UNPOWERED, WARMING or UNCALIBRATED, and only OK values drive automatic actions.

Reading contract

Use a fixed-size record per metric containing:

Field	Purpose
sensor ID, metric ID	Which device and which quantity (e.g. BME280 / humidity).
value and unit	The converted value; the raw/ADC value too where relevant.
acquisition interval start / end	When the measurement began and finished (monotonic time).
last-good timestamp	When the last valid value was obtained; kept separately from the current result.
sample sequence	Increments per new sample, so duplicates and gaps are visible.
power epoch	The rail epoch the sample belongs to (section 14).
configuration revision, calibration revision	Which configuration and calibration produced the value.
health and quality flags	See the table below.

Keep clock time optional; age and deadlines always use a monotonic timestamp, not NTP time.

Health / quality	Meaning and policy
STARTING / WARMING	No valid result yet, heater/restart settling, or filter not filled. No numeric threshold action.
OK	Fresh valid sample with declared calibration/units. Eligible for configured numeric rules.
STALE	Previously good data has exceeded its age limit. Show age; inhibit dependent automatic actions.
FAULT / TIMEOUT	Transport, CRC, range or conversion error. Preserve last-good separately.
UNPOWERED	Supply unavailable or its epoch changed during measurement. Do not interpret voltage decay as a physical value.
UNCALIBRATED / DEPENDENCY_INVALID	Acquisition works but physical units or compensation are not trustworthy. Publish raw data with explicit quality.

An analog input alone usually cannot prove probe presence: disconnected inputs can float or sit at plausible voltages. Diagnose rail/range/noise anomalies, but do not promise every unplug event becomes FAULT without additional hardware evidence.

Latest snapshot

The active acquisition lease holder is the only writer to a length-one snapshot queue. Publish a consistent copy after each completed sensor update and at the frame boundary using xQueueOverwrite(). BLE, MQTT and Safety use xQueuePeek(..., 0): they never wait for a future measurement or consume one another's snapshot. A snapshot is a coherent record copy, not a simultaneous measurement of all sensors.

Mode/status has its own single-writer PowerCoordinator mailbox. Consumers combine it with the acquisition record and the current rail epoch. If an external-5 V sample belongs to an obsolete epoch, expose it as unavailable even if acquisition has not yet published the next snapshot. Never send a retained last-good value as a new observation.

Freshness limits

Initial stale limits: **BME280 1 s**, **ordinary 1 Hz readings 3 s** (this includes the SGP30). After every SGP30 initialization its outputs stay WARMING for 15 s, during which it returns fixed values (400 ppm CO2eq, 0 ppb TVOC) that must never be published as measurements. Limits are configuration, not hard-coded library behavior. [E13]

TDS temperature compensation uses a valid DS18B20 sample and records its age/revision. If unavailable, provide raw voltage or an explicitly marked uncompensated estimate; do not silently substitute 25 degrees and call the result calibrated. [L9, L11]

SGP30 humidity compensation, when enabled, uses absolute humidity derived from a fresh BME280 temperature and humidity sample. If that sample is stale, stop updating the compensation and report DEPENDENCY_INVALID rather than keep an old value silently. The baseline status (restored, learning, valid) is reported separately from the readings. [E13]

11. Synchronization and queue contracts

IN SHORT

Each kind of data has its own channel with a fixed capacity and a defined behavior when it is full. Latest values are overwritten, alarms are sticky so they can never be lost, commands are rejected with “busy” rather than dropped silently, and logs are dropped and counted rather than blocking acquisition.

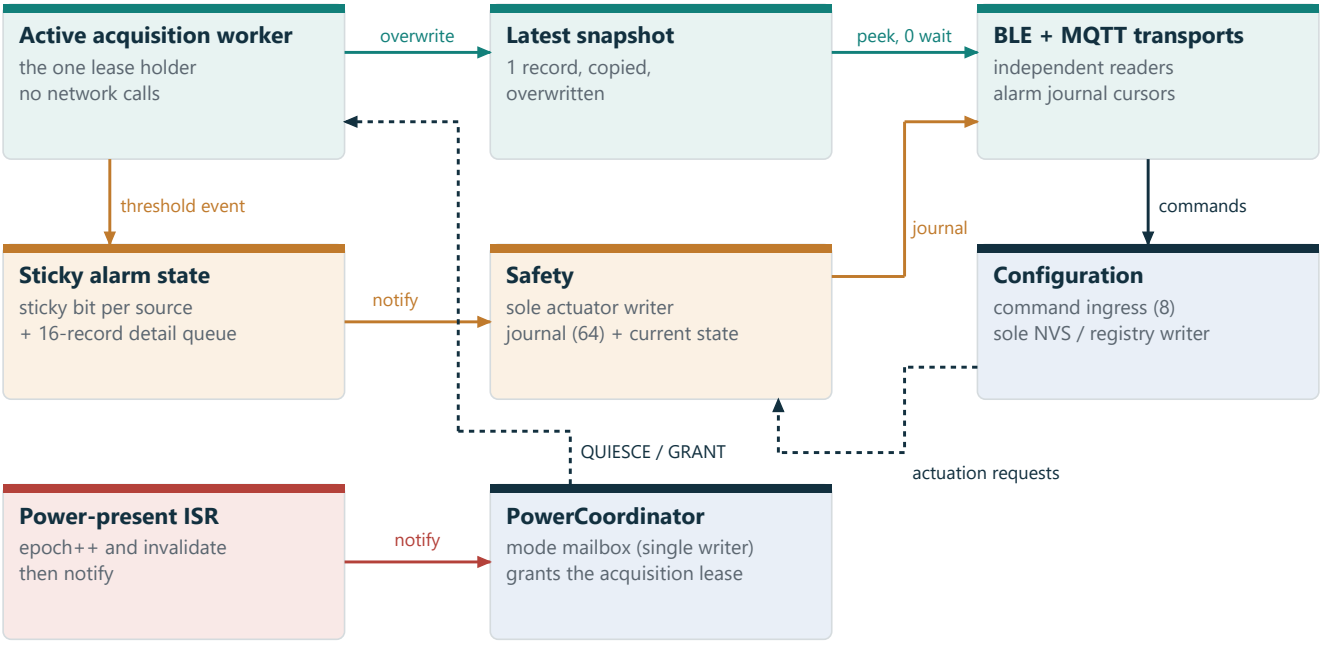


Figure 6 – Data paths between tasks. Solid = data; dashed = control.

Channel	Owners / capacity	Full or slow-consumer behavior
Latest snapshot	Acquisition → peek readers; 1 record	Overwrite old snapshot. Historical samples may be skipped by design.
Pending alarm store + notification	Acquisition/ISR adapters → Safety; fixed per-source entries	Sticky event bit plus first/worst observation and count. A later normal sample cannot erase an unhandled critical event.
Alarm detail queue	Producers → Safety; 16 records	Zero-wait send. On full, increment overflow count and preserve sticky alarm state.
Alarm journal	Safety → independent BLE/MQTT cursors; 64 events	Drop oldest detail with gap indication. Current latched alarm state remains available.
Command ingress	Transport → Configuration; 8 small records	Reject busy; do not silently drop or acknowledge success. Separate actuation request path to Safety.
Mode/lease control	PowerCoordinator ↔ workers; one grant each	ACKs carry transition generation. A stale ACK never permits another bus owner.
Telemetry backlog	MQTT owner; 60 one-second summaries	RAM only, drop oldest, expose count and original timestamps. No unlimited offline history.
Serial log	All tasks → Logger; bounded, e.g. 32 entries	Drop low-priority logs and count loss; never block acquisition.

Critical invariants

- No two acquisition workers hold the lease. Safety alone writes relay outputs. WiFiMqtt alone calls its MQTT client. Configuration alone commits NVS and registry updates. BLE callbacks copy validated bounded input and return; they do not touch sensor objects.
- Use FreeRTOS queue copying for plain fixed-size messages. A 4096-byte BLE frame uses a fixed buffer pool with explicit allocate/submit/release ownership; no stack pointers, borrowed String buffers or JsonObject views cross task boundaries. Commands carry request ID, authenticated connection/session generation and expiry.
- Use a short portMUX-protected rail record and sticky-alarm metadata where ISR/task coordination requires it. No I/O, blocking, allocation or large JSON copies occur inside a critical section. *volatile* is not cross-core synchronization. Hardware driver mutexes stay inside the single-owner boundary; an application mutex is not held across network calls.
- Separate local alarm handling from delivery. One queue read by Safety and Net would distribute events between consumers, not broadcast them. Neither a 16-entry queue nor MQTT can guarantee lossless delivery forever. The invariant is that local safety state survives detail-queue saturation, while telemetry reports any lost history.

WHY THIS REMOVES THE "BLE WAITS FOR A SENSOR" PROBLEM

BLE never asks a sensor for a value. It peeks the latest snapshot (zero wait), sends it with its age and quality, and if the user wants a fresher value it submits an asynchronous refresh request. A slow or failed sensor therefore shows up as an older age or a FAULT flag – never as a blocked BLE task or a cross-core deadlock.

12. Safety policy and control arbitration

IN SHORT

Safety is a small state machine on core 0 that owns the relay. It handles critical events before anything else, turns stale or missing data into alarms of their own, and never lets a remote command override a local interlock. Automatic actuation stays disabled until the relay wiring and loads are verified.

Safety is a small state machine, not a second sensor reader. Acquisition evaluates a pure threshold function against each eligible sample and submits an event immediately. Safety maintains latched alarms, stale/missing-data alarms, control leases and actuator maximum-on timers. It checks pending critical events before ordinary commands on every wake.

Processing order on every wake

1. Inspect power validity and transition generation; invalidate unavailable inputs and reject commands for unpowered outputs.
2. Drain sticky critical events and process pending alarm records. Take the configured local action immediately; network reporting is secondary.
3. Evaluate freshness/dependency loss even if no new sample arrived. An acquisition stall must not keep a pump enabled using old data.
4. Apply bounded authorized commands that pass local interlocks and expiry checks. Return accepted/rejected/applied status through the requesting transport.
5. Publish actuator state, alarm state and diagnostics; block until another event or a bounded health-check deadline (100 ms).

Rule definition

Each rule contains: source metric, valid range/quality requirements, trip threshold, clear threshold, debounce or persistence duration, severity, latch policy, target actuator action and explicit reset conditions. Hysteresis prevents chatter. A severe limit may trip on one valid sample; do not add a long averaging/debounce window that contradicts its response target.

```
Example rule (illustrative values only - real limits are not yet commissioned)
  source      : BME280.temperature, quality must be OK
  trip / clear: 40.0 C / 37.0 C            (hysteresis 3 C)
  persistence : 1 valid sample              (severe limit)
  severity    : CRITICAL, latched
  action      : configured relay state, max-on timer applies
  reset       : valid sample below clear threshold + configured reset policy
```

Faults and environmental alarms are distinct. An unpowered pH module may output zero volts; that is UNPOWERED, not an extremely acidic reservoir. Removing power must not erase an already latched real alarm. Clearing requires a valid recovery observation and the configured reset policy.

Actuator policy is a commissioning gate

The drawing shows one SRD-05VDC relay whose coil is fed from external 5 V and switched low-side by an IRLZ44N on GPIO 15: active-high, with a 10 kΩ gate pull-down and a 1N5822 flyback diode. It is therefore never available in low power. Still to establish: which contact (normally open or closed) is used, the driven load and its power source. The inherited code drives its relay active-low; do not port that polarity.

Default automatic actuation stays disabled until this mapping is verified. Electrical de-energization is the provisional boot/reset state; whether that is safe for circulation, aeration or valves must be decided per load. If circulation must continue on battery, its electrical branch and control requirement must be resolved together. Firmware cannot restore an unpowered actuator.

Remote control never overrides an active local interlock. Use maximum-on time and command expiry; do not replay stale actuator commands from an offline telemetry queue. [L2, L5, L7]

13. Power-mode state machine

IN SHORT

A dedicated “external power present” input drives the mode. Losing it immediately invalidates the external-5 V sensors, then the system moves through ENTERING LP to LOW POWER once every task has acknowledged. Coming back requires 500 ms of stable power and per-sensor warm-up. If any task fails to acknowledge, the system reports DEGRADED instead of guessing.

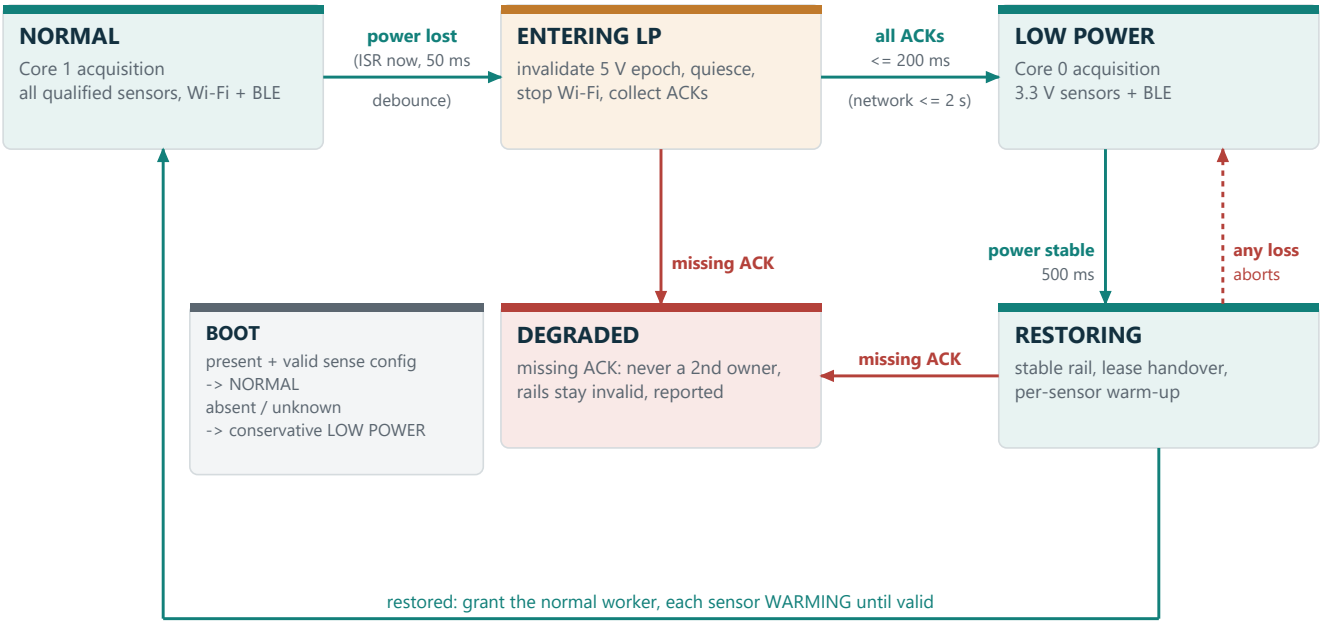


Figure 7 – Power modes. Any loss during RESTORING aborts restoration; a failed acknowledgement leads to DEGRADED.

Detection contract

The external-power-present signal is GPIO 8: a 220 Ω over 2 × 220 Ω divider from the external 5 V feed, not from ESP32 VDD or the battery-backed source bus. At 5.00 V it gives ≈ 3.33 V. At nominal 3.3 V, V_{IH} minimum is 0.75 × VDD (2.475 V) and V_{IL} maximum is 0.25 × VDD (0.825 V). Intermediate levels, for example from a sagging feed, are not a third reliable mode. The divider has no hysteresis, so debounce the mode in firmware. GPIO 8 is also ADC1_CH7, so the actual levels and decay can be measured during commissioning. [L6, L14]

The drawings have no rail-good signal per branch. External-source presence is not proof that the filtered 5 V branch feeding pH and TDS has settled. Use verified settle timing and report that limitation. Until the sense behavior has been measured on the board, keep the power-transition feature disabled and the build status non-production.

Entering low power

1. On an edge, invalidate the external-5 V epoch immediately (in the ISR).
2. PowerCoordinator starts a proposed 50 ms debounce, asks acquisition to quiesce at its next bounded operation boundary, requests WiFiMqtt shutdown and asks Safety to apply unavailable-actuator policy. These operations do not wait for MQTT delivery. Logger drains a bounded amount and blocks.
3. After the active worker cancels/finishes operations, sets powered-off interface pins safe and releases the lease, PowerCoordinator grants the low-power worker the new generation.
4. BLE announces LOW_POWER only when mandatory transition acknowledgements are complete; otherwise report TRANSITION/DEGRADED. Low-power acquisition keeps the four 3.3 V sensors running at their declared periods.

Returning to normal

1. Require stable external power (initial proposal: 500 ms) and verified rail settling.
2. Quiesce the low-power worker, invalidate pending conversions/filter windows as needed, then grant the normal worker the new generation.
3. Reinitialize restored devices and mark each WARMING until individually valid.
4. Request Wi-Fi reconnect asynchronously. Normal sensing must not wait for a broker.

Boot and failure rules

Boot with absent/unknown power takes the conservative low-power policy; invalid sense configuration prevents a production-ready status. A new falling edge during recovery aborts recovery. Transition ACKs have a proposed **200 ms** deadline, except network shutdown's separately bounded **2 s** deadline. Missing ACKs enter DEGRADED; they never justify granting two owners or falsely reporting complete shutdown.

14. Rail races and safe ownership handover

IN SHORT

Removing external power can cut the 5 V sensors at any instant, even in the middle of a reading. Every 5 V measurement therefore records the rail epoch when it starts and is accepted only if the epoch is unchanged when it ends. Moving the sensors between the two acquisition workers follows a strict quiesce → release → grant protocol so two tasks never drive the same bus.

Why a boolean is insufficient

Checking rail5vValid before and after a read misses a loss-and-return sequence if the flag becomes true again between checks. Use a monotonically changing rail epoch with the validity flag. The ISR increments the epoch and clears validity on every observed sense edge; only PowerCoordinator can restore validity after stability/rail checks. Debounce the mode, not the initial invalidation.

```
Before 5 V operation: copy {valid, epoch}; refuse if !valid.
Start bounded measurement; tag pending conversion with epoch.
On completion: copy rail state again.
Accept only if still valid AND epoch is unchanged.
Check epoch again when Safety/transport consumes the result.
Otherwise: discard numeric result; publish UNPOWERED/TRANSITION.
```

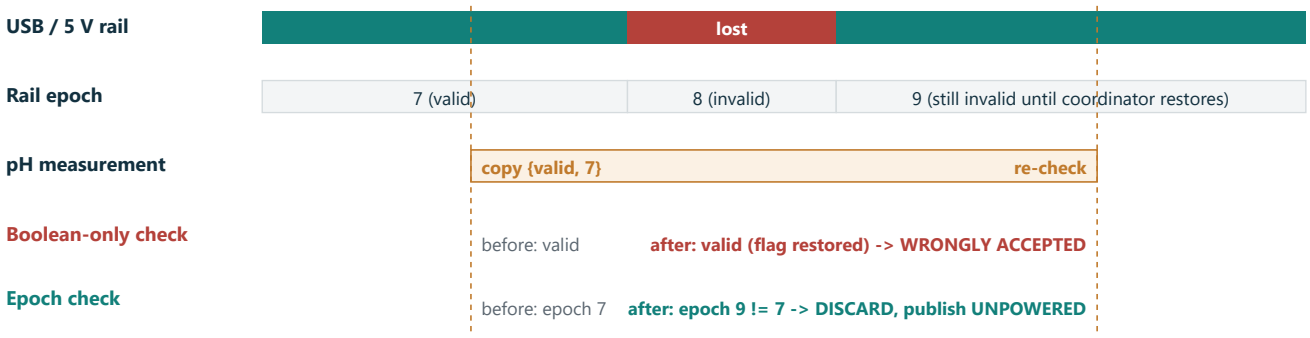


Figure 8 – Power drops and returns during one pH filter window. A boolean-only check can accept the reading; the epoch check rejects it.

WORKED EXAMPLE – USB UNPLUGGED DURING A PH READ

1. Rail state is {valid, epoch 7}. AcquisitionNormal copies it and starts the pH filter window.
2. The USB cable is pulled. The external 5 V feed and its filtered branch collapse, so the pH module output decays toward 0 V (the inherited pH formula would read about pH −8.8 here). The ESP32-S3 keeps running from the battery path through the source bus.
3. The power ISR runs on core 0: validity = false, epoch = 8, notify PowerCoordinator. Nothing else happens in the ISR.
4. The cable reconnects a few milliseconds later: the ISR runs again, epoch = 9, validity stays false. Only PowerCoordinator may restore it, after 500 ms of stable power.
5. The pH window ends. The worker re-copies the rail state: epoch 9 ≠ 7, so the numeric result is discarded and pH is published as UNPOWERED. No pH alarm is raised; any alarm latched before the cut stays latched.

Short critical sections make rail-state copies coherent across cores. Never keep the rail lock while converting or reading a bus. Keep ISR operations audited and ISR-safe; a std::atomic declaration alone is not proof of interrupt/cache safety. Queue messages and peripheral completion events include the transition generation so delayed callbacks cannot resurrect an old result.

LIMIT OF THE GUARANTEE

This protects against *observed* transitions. If physical voltage collapses before the sense interrupt is serviced, software cannot prove that a sample taken in that interval is valid. Scope the power-sense/rail relationship and interrupt delay; use rail-good hardware for a tighter guarantee. Firmware is not a substitute for adequate brownout and supply design.

Lease protocol

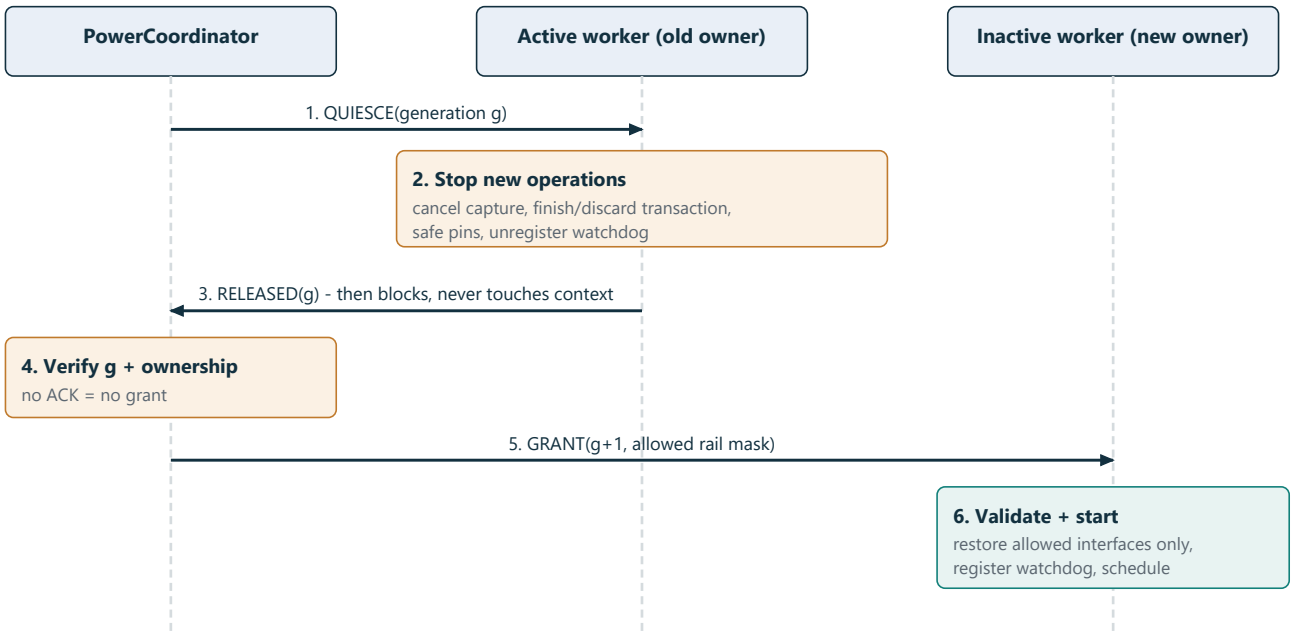


Figure 9 – Handing the sensor buses from one acquisition worker to the other.

1. PowerCoordinator sends QUIESCE(generation) to the current worker. The worker stops launching operations and wakes from bounded waits.
2. Worker cancels capture, completes or discards transactions, disables unsafe output driving, unregisters its active watchdog obligation and relinquishes context access.
3. Worker sends RELEASED(generation) and blocks awaiting a future grant. It must not touch the context after RELEASED.

4 Coordinator verifies generation and ownership, then issues GRANT(new generation, allowed rail mask) to the other worker. No ACK means no grant.

5 New worker validates pending state, restores only allowed interfaces and begins scheduling. ISR/capture handlers deliver to fixed queues or generation-tagged handles, never stale task pointers.

For HC-SR04, hold TRIG low during shutdown, then use the verified non-backfeeding pin configuration. Disable pull-ups or active driving into any unpowered board. Drive GPIO 15 low before release; the gate pull-down then keeps the relay driver off. Peripherals/interrupts with core-local allocation rules must be installed and released from their required core or use a fixed interrupt broker. Validate the chosen HAL before enabling the handover feature. [E2, L5–L6]

15. Low-power scope and framework choice

IN SHORT

The first low-power mode switches off Wi-Fi/MQTT and the external-5 V sensors, keeps BLE and the four 3.3 V sensors, and runs everything on core 0. Core 1 is idle, not powered off: an ESP32-S3 cannot turn a core off at runtime. Automatic light sleep is not available with the chosen Arduino build; measure the real current before promising battery life.

Baseline build

Use the demo's pinned PlatformIO espressif32@6.12.0 platform, which selects the Arduino 2.0.17 package family, with the esp32-s3-devkitc-1 board definition configured for the N16R8 (QIO flash, OPI PSRAM, 16 MB partition table). Keep NimBLE-Arduino 2.5.1 and ArduinoJson 6.21.5 pinned to preserve the demo protocol baseline; lock additional dependencies only after a compile and API check. This is a reproducibility choice, not a claim that these are the latest releases. [E5, L3]

The Stage 1 build resolved framework-arduinoespressif32 3.20017.241212. The upstream Arduino 2.0.17 ESP32-S3 sdkconfig has CONFIG_PM_ENABLE disabled, a 1000 Hz tick, dual-core FreeRTOS, Bluetooth controller affinity on core 0, loop() on core 1 and a 5 s task watchdog. Confirm the same values in the resolved package before relying on them. [E6]

What the first low-power mode can deliver

Item	Normal mode	Low-power mode
Wi-Fi / MQTT	On	Off (Wi-Fi disconnect and WIFI_OFF, performed by the WiFiMqtt service)
BLE	On	On
Sensors	All qualified sensors	BME280, SGP30, DS18B20, DFR0026 only (retained 3.3 V supply)
Acquisition	AcquisitionNormal, core 1	AcquisitionLowPower, core 0
Serial logging	On (core 1)	Blocked; diagnostics via BLE
CPU frequency	160 MHz (proposed)	80 MHz is an optimization candidate: it must pass BLE, bus timing and deadline tests; switch only at a quiescent boundary and check the API result
Core 1	Application work	Application tasks blocked; idle (WAITI). Scheduler interrupts and SDK work may still execute

There is no supported runtime application switch from a dual-core FreeRTOS image to a unicore image. CONFIG_FREERTOS_UNICORE is a build option. Core 1 can be mostly idle with its application tasks blocked, but scheduler interrupts and SDK work may still execute. “Single core” must be reported as application affinity, with

measured current savings, not as power-gating proof. [E2]

Deeper sleep is a later decision

Automatic DFS/light sleep requires a compatible SDK built with power management and tickless idle. Defining a compiler macro in application code does not rebuild Arduino's precompiled SDK. BLE light sleep also depends on the Bluetooth low-power clock source; check the ESP32-S3 options in the IDF guide before assuming an external 32 kHz crystal, which the current drawing does not show. [E3, E6]

If measurements show the baseline cannot meet battery targets, evaluate ESP-IDF or Arduino as an ESP-IDF component with a controlled sdkconfig. Keep driver/domain interfaces portable for that route. Do not promise connected BLE during manual deep sleep; deep sleep is incompatible with the requested continuously available link. [E9]

Measure whole-board current in normal, Wi-Fi-off dual-core, single-core application, and reduced-frequency modes. Regulator losses, relay coils, sensor LEDs and heater loads matter. The SGP30 heats its sensing element continuously and stays on the battery branch, so measure its share explicitly. This plan makes no battery-runtime claim.

16. Sensor contracts and bring-up decisions

> **IN SHORT**
> Every driver has the same six-function interface and never hides a long delay. Each sensor is “done” only when it has passed its own evidence list, not when it compiles. The SGP30 needs particular care: it must be measured every second without gaps, and its first outputs and its baseline need explicit states.

Each driver exposes:

Function	Contract
init(config)	Probe and configure the device; report identity/status.
start(now)	Begin a measurement or conversion without waiting for it.
poll(now)	Return PENDING / READY / error and a next-ready deadline. No hidden long delay.
readResult()	Return the typed, bounded result of the finished measurement.
cancel()	Abandon an in-progress operation (used by quiesce).
preparePowerOff()	Put pins and device into the safe, non-backfeeding state before a rail is cut.

A driver can wrap a proven manufacturer library, but its worst blocking operation must be documented and bounded.

Driver	Initial implementation contract	Completion evidence
BME280	Verify chip ID; forced mode, oversampling ×1 T/P/H; read compensation coefficients; use Bosch compensation; 12 ms planned completion budget.	Known reference comparison; missing device, stuck bus and conversion timeout; timestamps and 250 ms schedule.
DS18B20	External 3.3 V supply; select stable ROM ID; 12-bit asynchronous conversion, 750 ms limit; scratchpad CRC.	Missing/CRC error handling; first-conversion state; no blind acceptance of the 85 °C power-on value. A real 85 °C result is valid after confirmed conversion.
DFR0026	Read calibrated ADC millivolts/raw; characterize filter and saturation.	Dark/bright response and unplug behavior characterized; lux only after reference-meter calibration.
HC-SR04	10 µs trigger; bounded capture of ECHO; 30 ms timeout; measured mounting geometry.	No echo, stuck high/low and invalid distance; empty/full tank mapping checked physically.

Driver	Initial implementation contract	Completion evidence
pH-4502C	ADC calibration → divider correction → measured pH fit; timestamp spaced filter window.	At least two appropriate buffer points, independent check buffer and settling/noise measurements. Exact board documentation remains weak.
SEN0244 TDS	Preserve raw voltage; valid sample-count filter; calibrated transfer with fresh water temperature.	Reference solution and temperature tests; invalid dependency and saturation handling. Do not equate TDS estimate with calibrated EC.
SGP30	Initialize after every power-up or reset; one measurement per second from the absolute frame; TVOC and CO2eq with a CRC check on every word; restore a stored baseline; optional absolute-humidity compensation from the BME280.	Fixed initial values marked WARMING; CRC error and missing-device handling; baseline save/restore across a reset; clean-air and known-VOC-source checks; CO2eq labelled as an estimate.

The SGP30 needs a steady cadence and a baseline

The SGP30's on-chip compensation assumes a measurement command about once per second. For the first 15 s after initialization it returns fixed values, and its baseline must either be restored from storage or learned over a long run before values are fully compensated [E13]. Schedule it from the same absolute frame in both power modes (it stays on the retained supply), count missed seconds, store the baseline through Configuration at a rate-limited interval, and report baseline status beside the readings.

Report TVOC in ppb and CO2eq in ppm as an estimate, never as a CO2 measurement. The SGP30 chip itself is a 1.8 V part; confirm that the fitted module regulates its supply and shifts the I2C levels from 3.3 V. [E13, L7]

NEW IN REV 3 – MQ-7 REMOVED

Rev 1–2 planned an MQ-7 CO sensor with a heater-phase state machine. It is no longer part of the hardware (Redesign Specification Rev 5). In the firmware, SENSOR_MQ7 and METRIC_CO_SIGNAL were replaced by SENSOR_SGP30, METRIC_TVOC and METRIC_CO2EQ (Snapshot 528 → 584 bytes), and drivers/mq7.* by the drivers/sgp30.* stub with its test_sgp30 environment. L10 is kept only for traceability.

Analog disconnected-probe detection and pH module thresholds require board-specific evidence. A successful compile or plausible serial number does not establish calibrated accuracy. [L8–L13]

17. BLE, MQTT and configuration boundary

IN SHORT

Keep the existing BLE protocol (UUIDs, framing, secure pairing) but make it carry real health and power information. Wi-Fi/MQTT retries with backoff and never restarts the board. Configuration changes are validated, saved as a versioned whole and applied only at a safe moment.

Preserve the useful protocol

Keep the existing service/characteristic UUID map and two-byte little-endian length framing, with a 4096-byte maximum JSON payload. Retain authenticated access, LE Secure Connections/bonding and request IDs. Implement physical commissioning and owner-recovery policies; the demo currently has no time-limited commissioning window. Do not claim the demo's ownership scheme alone resolves all production authorization needs. [L4]

State continues every 2 s and on a committed mode change; telemetry may remain every 3 s while subscribed, as in the app contract. Both consume available records immediately. Include sample age, health, power availability, sequence and configuration revision. No valid numeric value is sent for UNPOWERED. Omit battery percentage until it is actually measured and estimated.

The demo advertises pH/TDS/level as low-power essentials and pauses climate/light: reverse this policy to match the physical rails. Its climate telemetry carries only one metric; plan support for BME280 humidity/pressure, SGP30 TVOC/CO2eq and baseline status, water temperature, unavailable readings and schema negotiation in Protocol.py, Catalog.py and the UI. Treat this as a coordinated protocol revision, not a promise of unchanged-app compatibility.

Bounded transport services

BLE. Callbacks validate length/session, copy to a fixed pool and enqueue without blocking. BleTransport serializes complete frames per connection, prioritizes results/alarms over periodic telemetry and stops sending after disconnect/session change. Partial frame failure resets framing through an explicit reconnect/resync policy. A notification does not prove the laptop processed an alarm; use event IDs and acknowledgement if that guarantee is required.

Wi-Fi/MQTT. WiFiMqtt owns its connection state machine. Use finite socket/connect timeouts and exponential retry backoff, initially 1, 2, 4, 8 seconds up to 60 seconds with jitter. No broker failure restarts the sensing system. Power-stop requests must preempt retry waiting and have a bounded shutdown path. Verify client calls actually yield and honor deadlines; move to an asynchronous MQTT client if they cannot.

Configuration transactions

Retain get_devices / set_wifi / add_device / delete_device through validated commands. Board pin mappings remain fixed initially; runtime logical instances may only use supported predeclared slots. Request quiescence before changing a driver's context; never delete an object from a BLE callback. Validate, stage, persist a versioned configuration, apply it at a safe boundary, then acknowledge the actual outcome. Recovery loads the last valid committed version.

NVS writes and flash erase can affect both cores. Rate-limit writes and schedule a controlled maintenance window with reported timing status; test its worst-case pause. Credentials are provisioned and never logged. Production remote actuation requires authenticated broker access and authorization; preserve legacy MQTT topic shapes only through an adapter.

18. Fault tolerance, deadlines and recovery

IN SHORT

Every failure has a defined behavior: mark the data, keep sensing and local safety going, report it, retry later within bounds, and reset only as a last resort with the reason recorded. Watchdogs prove that each task is making progress, not merely that the CPU is alive.

Failure	Required behavior
I2C missing device / stuck bus	Bounded transaction timeout; mark affected readings FAULT; schedule limited bus recovery under sole ownership. No repeated blocking retry loop. A stuck bus affects both the BME280 and the SGP30.
SGP30 CRC error / missed second	Reject the word, count the gap; after repeated gaps re-initialize and return to WARMING.
DS18B20 CRC / no presence	Reject result, retain last-good timestamp separately, retry at a bounded later deadline.
Analog saturation / unknown calibration	Expose raw value and quality; suppress invalid derived-unit alarm decisions.
Power edge during conversion	Discard by epoch; safe interface configuration; no false numeric alarm.
Broker or access point absent	Continue sampling and local safety; BLE reports network state; bounded backoff/backlog.
BLE congestion / disconnect	Drop superseded telemetry; preserve current alarm state; reject expired session commands.
Alarm / command queue full	Sticky local alarm persists; count lost detail; command sender receives busy/error rather than false success.
Acquisition deadline miss	Record lateness and sensor duration; skip expired work rather than burst-catching up; set degraded timing status.
Transition ACK missing	No second owner; keep unavailable rails invalid; local safe policy; supervisor diagnoses and escalates.
Task stalled / heap exhausted	Safe outputs where possible, persistent reset reason at next boot, bounded restart strategy; no endless restart loop.

Watchdog design

Use SDK task/interrupt watchdogs according to the pinned version. Each monitored task resets its own watchdog after completing meaningful work; Supervisor must not blindly feed it on behalf of stalled tasks. Idle subscription catches CPU starvation; a separate monotonic progress timestamp catches a task blocked forever while idle still runs. [E10]

Inactive acquisition workers deliberately block forever and are excluded from active-work progress deadlines. Update expected-task membership as part of the acknowledged lease transition. Safety's timed wait and the network state machine report progress even when there is no sensor event or broker connection. Start with a five-second task-watchdog budget only if every underlying call is proven shorter; a timeout counter and recovery attempt must precede reset where possible.

Bounded memory and logs

Allocate queues, pending-alarm entries, frame pools and sensor contexts once. Check every creation result. No dynamic JSON construction in acquisition or ISRs. Measure minimum free heap, largest free block and stack high-water marks during BLE/MQTT overlap. The initial stack table is not a memory-fit proof.

Rate-limit serial printing through Logger, with a short write timeout. In low power the logger acknowledges quiescence and blocks; diagnostics remain available through BLE. Backlog detail may be lost on power/reset, but current alarm and fault status must be reconstructible from local state. Do not write every sample to flash.

19. Implementation stages and acceptance

IN SHORT

Build sensor-first: the shared data model and one driver at a time, each with its own test environment. Then the scheduler and Safety with fake data, then BLE/MQTT, and only last the power-switching machinery. Each stage has measurable acceptance tests.

Sensor-first development sequence

1. Create firmware/ with the board profile, common Reading model and a small RTOS bring-up harness. Pin dependencies and inspect the resolved SDK configuration; compile for the actual ESP32-S3 board.
2. Implement BME280, then DS18B20, then DFR0026, then SGP30 (it reuses the BME280's I2C HAL). Each test_<sensor> environment links the production driver and one bring-up main; exclude production main so entry points cannot collide.
3. Implement HC-SR04, pH and TDS after each device's electrical prerequisites are satisfied. Calibration is a separate acceptance milestone from communication.
4. Integrate the acquisition scheduler, Safety and copied snapshots. Inject fake time/data to test ordering, stale values and alarm latching before network integration.
5. Add MQTT/BLE adapters and bounded commands. Update and test the desktop protocol for physical health and power policy.
6. Add power ISR, rail epochs, lease handover and measured power optimizations. Do not activate transitions until the hardware sense input and rail behavior are verified.

NEW IN REV 2 – D15: ONE ACQUISITION WORKER UNTIL STAGE 6

Stages 1–5 run a single acquisition task on core 1 that owns all buses directly. The second (low-power) worker, the lease/generation messages and the QUIESCE/RELEASED/GRANT handover of section 14 are added together in stage 6, when the power-sense input exists to drive them. The driver interface (start/poll/cancel/preparePowerOff) and the generation field in records are present from stage 1, so stage 6 adds the protocol without changing drivers. This keeps early sensor work simple without creating rework.

Test	Acceptance target / required evidence
Per-driver build	pio run -e test_<sensor> succeeds; recorded dependency versions and memory use. Build records are kept in firmware/README.md.
Sensor bench checks	Timestamped readings, reference comparison, startup and disconnect behavior; record what analog disconnection cannot detect.
One-second schedule	Logic-analyzer/monotonic trace: 1 s frame release, proposed ≤ 5 ms release jitter under specified healthy radio load. Record max and percentiles.
Temperature alarm	Inject a sustained threshold crossing at varied schedule phases: ≤ 300 ms software detection/action target and ≤ 10 ms from accepted sample to Safety action. Physical thermal response measured separately.
Safety under radio load (new)	Repeat the temperature-alarm test while Wi-Fi/MQTT publishes continuously and BLE streams telemetry. Record the sample-to-action distribution and the SDK task priorities seen in the trace. If ≤ 10 ms is not met, apply the D14 fallback and retest.
Power transitions	Drop/reapply power at every driver step, plus rapid bounce and loss-return within a read. No stale epoch accepted; no false pH/TDS numeric alarm; only one worker owns buses.
Network faults	AP/broker absent for 30 min; repeated BLE connect/disconnect and frame congestion; sensing continues, storage bounded and drop counters accurate.

Test	Acceptance target / required evidence
Queue overload	Fill command/alarm queues; verify busy results, sticky alarms, independent transport delivery and event-gap reporting.
Stall injection	Halt each active worker deliberately; progress watchdog detects it. Expected idle workers do not cause false resets.
Power/current	Compare modes at the supply input with identical workload; verify Wi-Fi off, BLE availability, readings from all four 3.3 V sensors and core-1 application blocking. Measure the SGP30's share of the battery current.
Endurance	24 h mixed-mode run with counters, max latency, heap/stack trends and reset causes; no unexpected watchdog reset or unbounded growth.

Commissioning gates: measured power-sense levels on GPIO 8, branch settle timing, relay contact/load policy, SGP30 module checks, analog ranges/calibration, and final trip thresholds. These gates do not prevent isolated BME280 driver work from starting next.

20. Decision log and remaining gates

IN SHORT

Eighteen recorded decisions, each with the alternative it rejects (D02 is kept but superseded by D16). Changing one of them means updating this table and the sections that cite it. The open hardware items below must be closed before the matching features are switched on.

ID	Decision	Rationale / alternative rejected
D01	New firmware directory; preserve old code	Selective porting gives a clean boundary without discarding useful protocol knowledge.
D02	ESP32-WROOM-32D remains target (superseded by D16)	Rev 1–2 decision, kept for traceability. The board changed to the ESP32-S3 in October 2026.
D03	Explicit Espressif FreeRTOS	Supported kernel and radio integration; no second RTOS or bare-metal replacement.
D04	One driver per sensor, thin main	Keeps sensor bring-up familiar; services own orchestration instead of a growing main.
D05	One serial acquisition owner	Removes peripheral races without creating one task and mutex set per sensor.
D06	1 s frame + 250 ms BME280 checks	Predictable baseline and shorter software blind interval; does not claim a hardware temperature interrupt.
D07	Core 1 acquisition, core 0 networking/Safety	Matches typical ESP32 radio allocation and avoids migrating safety state. Explicit pinning intentionally differs from Rev 4's conditional affinity guidance.
D08	Pre-created workers with generation lease	Avoids task-delete/recreate races and repeated allocation; costs one extra stack. Handover still requires peripheral-affinity validation.
D09	Latest snapshots, sticky safety state	Fast readers never wait for sensors; finite event queues no longer imply impossible lossless delivery.
D10	Power epoch, not only a boolean	Detects power loss and return during a pending measurement.
D11	Awake low-power baseline	Preserves BLE; automatic sleep requires a separate SDK/hardware decision.

ID	Decision	Rationale / alternative rejected
D12	Rail-based availability, not demo essential flag	pH/TDS/level cannot operate when their external 5 V supply is removed.
D13	Calibrated units only after evidence	pH fit, TDS compensation, light calibration and the SGP30 baseline/humidity compensation are explicit quality gates.
D14	Safety stays on core 0, verified under radio load	SDK radio tasks outrank Safety on core 0. Keep D07 but make the ≤ 10 ms target a radio-load test; fallback is Safety on core 1 in normal mode with a quiesce/grant handover. Rejected: assuming application priority 6 bounds latency.
D15	Single acquisition worker until stage 6	Early sensor work stays simple; interfaces already carry generation fields so the lease is added without driver rework. Rejected: building the full handover before the power-sense hardware exists.
D16	ESP32-S3-WROOM-1-N16R8 is the target (supersedes D02)	Hardware changed. Use only the GPIOs on the 22 October drawing; keep stacks, queues and ISR data in internal SRAM and use PSRAM only for measured bulk buffers. Rejected: keeping a WROOM-32D pin map.
D17	Project code in C on the Arduino-ESP32 framework	Migrated 16 October 2026 with layer rules, contracts and record sizes unchanged. C++ libraries only behind an adapter. Rejected: moving to bare ESP-IDF now (section 15 keeps that route open).
D18	SGP30 on the retained 3.3 V supply replaces the MQ-7	Per Redesign Specification Rev 5. It shares the I2C bus, needs no heater-profile hardware or 5 V divider and stays available on battery. It reports TVOC and an estimated CO2eq, not CO. Rejected: keeping the MQ-7, whose specified heater cycle needed hardware the module did not expose.

Open hardware and application decisions

Open item	Blocks
External-power sense on GPIO 8: measured levels, decay and bounce	Power ISR and low-power transitions (stage 6)
Per-branch rail feedback (not on the drawing)	Tighter rail-validity guarantee (section 14)
Measured outage/return behavior, including the source-selection transfer	Confirming the 50 ms debounce and 500 ms restore values
Relay contact use (NO/NC), driven load and load-safe state	Any automatic or remote actuation
SGP30 module: supply and logic levels, pull-ups, baseline storage policy	Trustworthy TVOC/CO2eq; battery current
pH / light ADC headroom against the S3's ~3.1 V range	Accuracy at the top of their ranges
All genuine trip/clear limits	Enabling Safety actions

The schematic now contains one relay, while the BLE catalog can describe four. Plan the initial board profile for only physically confirmed outputs. Adding more logical device entries cannot create more pins or supplies. Battery-backed circulation, if required, needs an explicit hardware and firmware requirement.

What is complete in this phase

Revisions 1–2 reviewed the source and drawings, checked manufacturer/SDK constraints and produced the architecture and staged validation plan. Since then the Stage 1 skeleton has been built in C for the ESP32-S3, the BME280 runs on S3 hardware and the pH driver is written (firmware/README.md). Revision 3 brings this document in line with the 22 October hardware. No calibration, power-transition behavior or power saving is claimed as validated. Circuit documents were not edited.

21. Local evidence register

IN SHORT

The files on which this document's claims rest. Paths are relative to the workspace root; line numbers refer to the files as inspected on 4 October 2026. L5–L8 and L14 were updated on 22 October 2026.

Ref	Source and relevant location
L1	User request and attached Pasted text.txt, attachment 7665e9d2-49da-4fbc-a419-b8acb71c80a6. Treated as a proposed draft; its unsupported guarantees were corrected.
L2	Hydroponics-Monitoring-System-master/Project Handover/Source Code/src/: main.cpp, connectToWifi.cpp, sensor.cpp, deviceManager.cpp, deviceFactory.cpp and all sensor/relay implementations inspected.
L3	Hydroponics-Monitoring-System-master/BluetoothControlApp/ESP-Demo/src/Main.cpp and ESP-Demo/platformio.ini. setLowPower at line 202; command queue creation at line 524.
L4	Hydroponics-Monitoring-System-master/BluetoothControlApp/FirmwareContract.md. Version 1 GATT, framing, security intent and simulation limits.
L5	Electrical System Study/Power Circuit Diagram (1).pdf, page 1. Charger, 3S battery and BMS, MP1584 5 V / 3.3 V regulators, IRLZ44N source selection, branch decoupling, ferrite-filtered pH/TDS branch, relay and per-sensor supplies.
L6	Electrical System Study/Pin Layout (1).pdf (Sensor ↔ ESP32-S3 Signal Pinout, 22 October 2026) and Pin Layout.drawio. Signal assignments, resistor and capacitor values, strapping/PSRAM/USB pins and unconnected comparator outputs.
L7	Electrical System Study/Redesign Specification.pdf, Rev 5 (16 October 2026), sections 2–7. Branch availability, signal table, SGP30 requirements, power sensing and task allocation.
L8	Electrical System Study/Datasheets/BME-280.pdf. Bosch Rev 1.23; sections 3, 4, 7.1 pin description and 9.1 measurement timing.
L9	Same Datasheets directory: DS18B20.pdf. Maxim Rev 6; page 3 conversion time, pages 5–7 conversion/power/alarm behavior, page 9 CRC.
L10	Same directory: MQ-7.pdf. Historical only: the MQ-7 was removed from the hardware in Rev 3 (D18).
L11	Same directory: SEN0244.PDF. DFRobot, pages 1–3 electrical limits, pages 4–7 filtering and temperature compensation.
L12	Same directory: HC-SR04.pdf. Supplied module guide, pages 2–5: trigger/echo, supply and measurement limitations. Generic module documentation; verify actual board.
L13	Same directory: DFR0026.PDF, pages 1–4, and PH-Sensor-4502C.pdf, pages 1–6. The pH file is a tutorial, not an authoritative manufacturer electrical specification.
L14	Same directory: ESP32-S3-WROOM-1U-N16R8.pdf. Espressif WROOM-1/1U datasheet v1.0: pin functions and ADC1 channels, PSRAM use of IO35–37, strapping pins (section 3.3) and DC characteristics (table 9).

Evidence hierarchy

1. The current request determines intent.
2. The current electrical diagram determines intended topology.
3. The pin drawing determines candidate signals.
4. Exact-part datasheets constrain implementation.
5. Existing code proves only what the inherited software does, and a demo proves only its simulated behavior.

Where these sources disagree, this report records a gate instead of silently selecting a convenient interpretation. A baseline hash manifest and post-generation preservation check accompanied the Rev 1 document QA.

22. Manufacturer and software references

IN SHORT

Primary online sources checked on 4 October 2026; the ESP32-S3 links and E13 were checked on 22 October 2026. Versioned links deliberately match the Arduino baseline where possible; the local sensor PDFs in section 21 remain the detailed sensor evidence.

Ref	Reference	Used for
E1	Espressif ESP32-S3-WROOM-1/1U datasheet	Processor, pins, electrical limits and power modes (online copy of L14).
E2	ESP-IDF v4.4.8 FreeRTOS SMP (ESP32-S3)	Affinity, queues/critical sections, deletion and unicore configuration.
E3	ESP-IDF v4.4.8 power management (ESP32-S3)	PM enablement, locks, tickless idle and BLE clock conditions.
E4	ESP-IDF v4.4.8 ADC (ESP32-S3)	ADC1/ADC2, Wi-Fi conflict, attenuation (0–3100 mV at 11 dB) and calibration.
E5	PlatformIO espressif32 v6.12.0 manifest	Framework package selection.
E6	Arduino-ESP32 2.0.17 ESP32-S3 sdkconfig	Upstream PM, tick and Bluetooth-controller configuration.
E7	NimBLE-Arduino 2.5.1 configuration	Library configuration entry point; inspect actual host affinity during integration.
E8	Espressif Bluetooth LE FAQ	Bluetooth LE low-power considerations.
E9	Espressif ESP32-S3 sleep modes	Connection-preserving modem/automatic sleep versus manual sleep.
E10	ESP-IDF v4.4.8 watchdogs (ESP32-S3)	Task/interrupt watchdog semantics.
E11	Bosch BME280 datasheet	Manufacturer source corresponding to the supplied local document.
E12	DFRobot DFR0026 product documentation	Analog ambient-light module characteristics.
E13	Sensirion SGP30 product page and datasheet	Measurement cadence, initialization values, CRC, baseline and humidity compensation. No local copy yet: add it to the Datasheets folder.

NEW IN REV 2 – FRAMEWORK VERSION NOTE

The ESP-IDF links above are the v4.4.8 documentation, but Arduino-ESP32 2.0.17 is built on an earlier 4.4.x release of ESP-IDF (4.4.7 according to its release notes). The architecture conclusions do not depend on the patch level, but every API, Kconfig option and default used in code must be checked against the IDF version actually resolved by the PlatformIO build (section 19, stage 1).

How to use this document during implementation

Keep the requirement IDs, decision IDs and acceptance tests visible during each sensor bring-up. Replace proposed timing budgets with measured maximums, record the exact library/build configuration, and update only the decisions affected by evidence. Preserve the distinction between reading a sensor, obtaining a calibrated value, and demonstrating an end-to-end safety response.

The first coding deliverable should be the shared minimal data model plus the BME280 driver and its isolated RTOS bring-up environment. It does not depend on choosing whether every old source file will eventually be retired.